

LAB 7

Multithreading

Due date : 06/04/2026

Lab Overview:

In this lab, you will write a multithreaded C program using the POSIX threads (pthreads) library. Several threads will update the same global counter. Since threads in the same process share global variables, two threads can accidentally update the counter at the same time. This can cause a race condition, where some updates are lost. To prevent this, you will use a mutex lock so that only one thread enters the critical section at a time.

Core Tasks:

1. Create multiple threads using `pthread_create()`.
2. Use command-line arguments for the number of threads and the number of increments per thread.
3. Protect the shared counter using `pthread_mutex_lock()` and `pthread_mutex_unlock()`.
4. Wait for all threads to finish using `pthread_join()` and print the final counter value.

Task Details:

1. Program input
 - a. Run your program using: `./lab7 <num_threads> <increments_per_thread>`
 - b. `num_threads` is how many threads your program should create.
 - c. `increments_per_thread` is how many times each thread should increment the counter.
2. Shared counter
 - a. Declare a global integer counter and initialize it to 0.

- b. Each thread should run `worker()` and increment the shared counter inside a loop.
 - c. The correct final value should be: `num_threads x increments_per_thread`.
- 3. Mutex synchronization
 - a. Initialize the mutex in `main()` using `pthread_mutex_init()`.
 - b. In `worker()`, lock the mutex before updating the counter and unlock it immediately after.
 - c. The counter update is the critical section because only one thread should modify it at a time.
- 4. Thread management
 - a. Create all threads using `pthread_create()`.
 - b. Wait for all threads using `pthread_join()`.
 - c. Destroy the mutex at the end using `pthread_mutex_destroy()`.
- 5. Race condition experiment
 - a. After the correct version works, temporarily comment out `pthread_mutex_lock()` and `pthread_mutex_unlock()`.
 - b. Recompile and run with large values such as: `./lab7 4 100000`.
 - c. Without the lock, the final counter may be lower than expected because threads can lose updates.

Building and Running:

From the `lab7` directory, run: `make`

- a. Example run: `./lab7 4 100000`
- b. Expected output: Final counter value: 400000
- c. Another example: `./lab7 8 50000`
- d. Expected output: Final counter value: 400000
- e. Valgrind:

```
make clean && make
valgrind --leak-check=full ./lab7 4 100000
```

Submission Requirements:

- 1. Files include
 - a. `lab7.c`
 - b. `Makefile`
- 2. Screenshots include:
 - a. Successful compilation using `make` with no errors.

- b. A correct run with the mutex enabled, where final counter equals `num_threads x increments_per_thread`.
 - c. A second correct run with different command-line arguments.
 - d. **Optional:** a run with the mutex commented out showing an incorrect/lower counter value.
 - e. **Valgrind** screenshot
3. Submit your lab 7 folder as a .tar.gz or .zip file to Canvas.

The naming convention of the zip/tar file while uploading to Canvas

UoID_duckID_LabX (an example is given below)

- First Name : Dewi
- Last Name: Yokelson
- Submission for: Lab7
- So the name of the zip/tar file should be:

Dewi_Yokelson_Lab7.zip